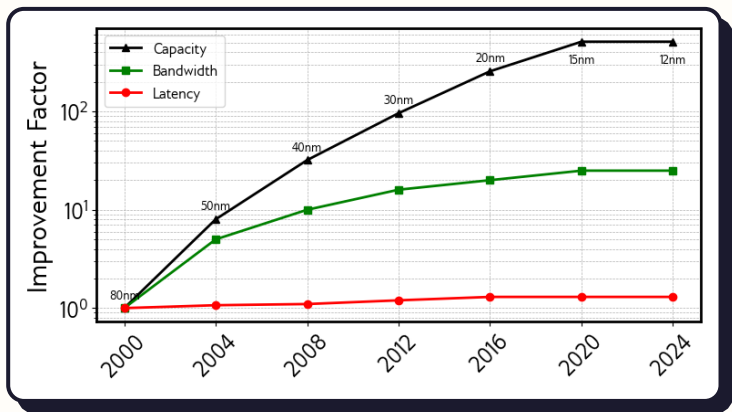


TierSizer: Counterfactual Reasoning for DRAM Sizing in Tiered Memory Systems

Sujay Yadalam, Saarth Deshpande, Divyanshu Saxena,
Venkat Arun, Neeraja J. Yadwadkar, Aditya Akella

DIMES 2026

Memory is eating the server budget



DRAM scaling has stagnated



DuckDB

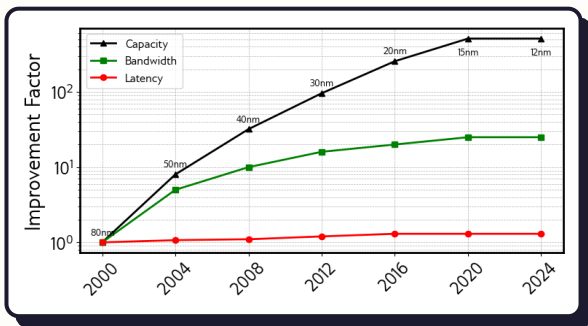
LMCache



Memory demand keeps growing

[1] O. Mutlu et.al. "A Modern Primer on Processing in Memory." on arXiv.

Memory is eating the server budget



DRAM scaling
has stagnated

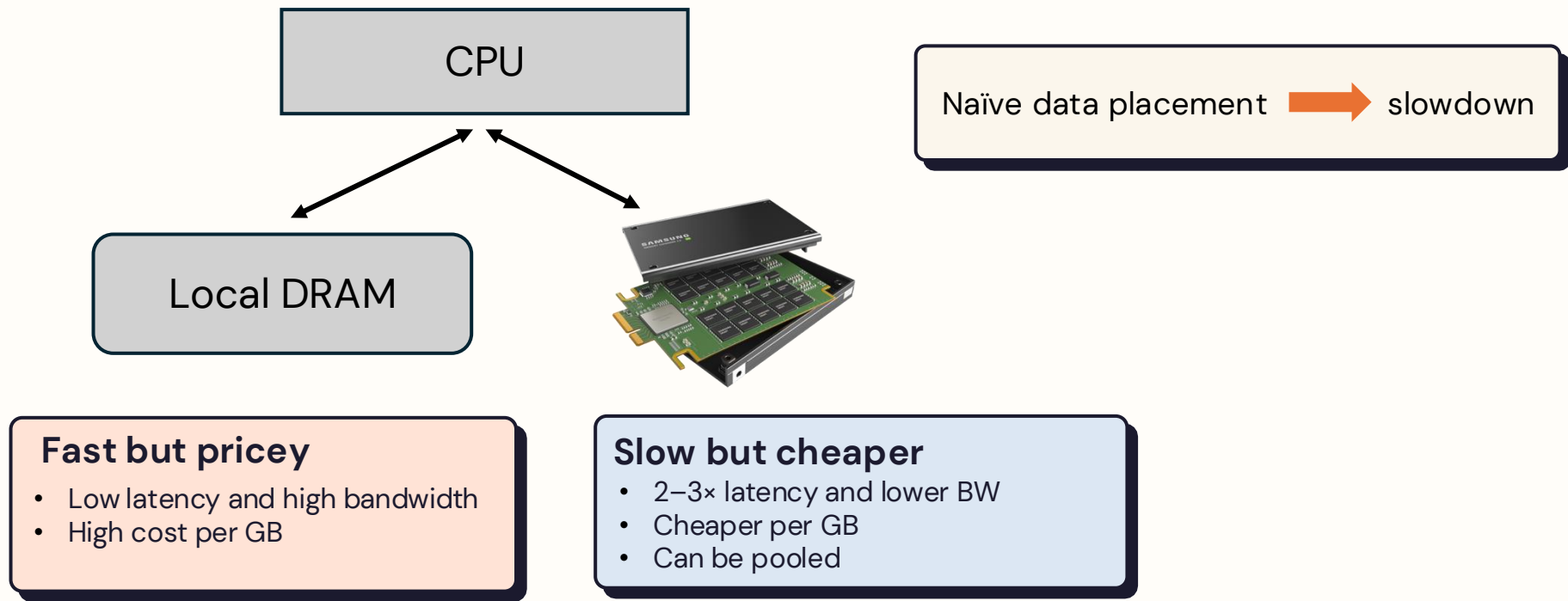
Memory demand
keeps growing

~40% of server costs goes to DRAM



[1] O. Mutlu et.al. "A Modern Primer on Processing in Memory." on arXiv.

CXL: cost-effective memory expansion



Tiering: Smart data placement

Key idea: Hot pages in DRAM, cold pages in CXL

Thermostat

AutoNUMA

HeMem

TPP

MENTIS

Colloid

FlexMem

HybridTier

ArtMem

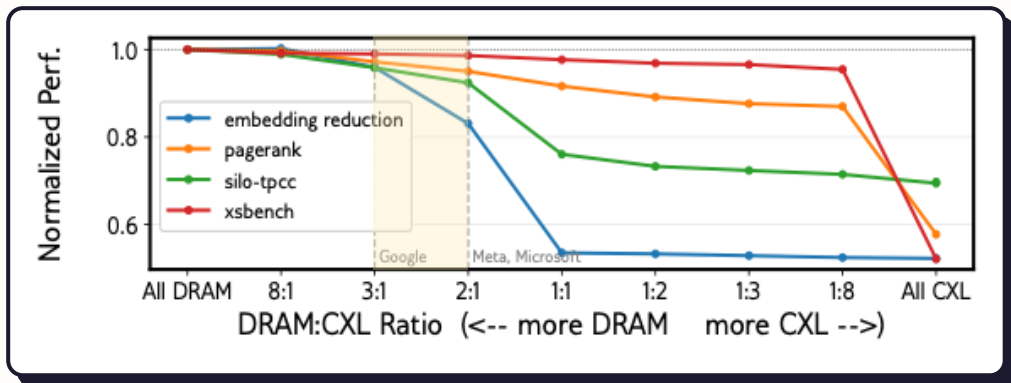
PACT

ARMS

...and many more

How big should DRAM be?

Performance across DRAM sizes



Google, Meta and Microsoft run 3:1 or 2:1 for everyone (shaded).

Every workload has a different knee

The performance drop can be drastic

THE KEY QUESTION

**Given a workload on a tiered system,
what's the minimum DRAM that still
hits its performance target?**

Too much DRAM wastes money. Too little misses the perf target.

Naïve approaches

Offline profiling

Sweep every DRAM size for every workload, ahead of time.

Expensive, often infeasible, and stale once the workload changes.

Online exploration

Shrink DRAM, measure, repeat until it hurts.

Small shrinks can degrade performance drastically hurting SLOs.

THE KEY IDEA

Ask “what if?**” without actually changing the DRAM size.**

Counterfactual analysis that predicts application slowdown at different DRAM sizes.

Key insight

Slowdown on CXL systems has two main sources:

#1 Increase in LLC-miss stalls

Accesses to CXL incur higher latency leading to increased stalls.

#2 Migration overhead

Access to a page mid-migration blocks thread execution; migration consumes bandwidth.

$$\Delta P_{D \rightarrow D'} = \frac{\Delta stalls + \Delta O_{mig}}{total\ cycles}$$

Estimating LLC-miss stalls

LLC stalls depend on:

1. Number of accesses to each tier (A_D, A_C)
2. Latency of each tier (L_D, L_C)
3. Memory Level Parallelism or MLP
4. Microarchitectural optimizations such as OoO, speculation (α_D, α_C)

$$\text{stalls} = \alpha_D \frac{A_D L_D}{\text{MLP}} + \alpha_C \frac{A_C L_C}{\text{MLP}} + \beta + \epsilon$$

Estimating stalls at DRAM size D'

$$\text{stalls} = \alpha_D \frac{A'_D L'_D}{\text{MLP}'} + \alpha_C \frac{A'_C L'_C}{\text{MLP}'} + \beta + \epsilon$$

Accesses

Predict which pages get demoted at D' . Their traffic, ΔA , moves from DRAM to CXL.

$$A'_D = A_D - \Delta A$$
$$A'_C = A_C + \Delta A$$

Latencies

Predict DRAM and CXL bandwidth utilization at D' . Use BW to index into BW-latency curves.

$$BW' \rightarrow L'_D, L'_C$$

MLP

We observe MLP barely moves between *nearby* DRAM sizes, so reuse current value.

$$\text{MLP}' \approx \text{MLP}$$

Error in predicting accesses

Accesses

Predict which pages get demoted at D'. Their traffic, ΔA , moves from DRAM to CXL.

$$A'_D = A_D - \Delta A$$

$$A'_C = A_C + \Delta A$$

Other sources of error

Latency and MLP averages

Hide variation within a measurement period.

Fine: Remain stable within small periods (~1s)

Latency estimated using BW-latency curves

Ignore read:write mix and access size.

Fine: we index them conservatively; the rest folds into ϵ .

Residual noise

Period-to-period drift, light interference, counter noise.

Fine: absorbed by ϵ , the margin we already carry.

All models are wrong, but
some are useful.
– George E.P. Box

WHY KEEP IT SIMPLE

Simple is enough

- Sizing needs only Δ stalls, shared errors cancel
- Few knobs: doesn't overfit, extrapolates well
- Needs little data, so it adapts fast
- A tiny LP refit every 30 s

2.27%

median stall error